

セルフホスティングの機能

評価器をLispで表す

情報システム学科 3年・選択・2単位

今日の到達目標

- 評価器が必要とする操作を列挙できる
- 対象言語の環境とホスト環境を区別できる
- Lisp製評価器の最小例を動かして拡張できる

前回との接続

ファイル、リスト、関数を使えるようになった。

今度は評価器自身をLispのプログラムとして書く。

足りない操作を、具体的な式から見つける。

対象言語とホスト

ホスト言語は評価器を動かす言語。

対象言語は評価器が処理する言語。

両方がLispでも、それぞれ別の環境と評価段階を持つ。

最小の目標

まず対象の数値と二項の `+` を評価する。

次に対象の名前、`quote`、`if` を追加する。

受入例を通してから関数へ広げる。

機能と用途

ホスト側の機能

評価器での用途

型の判定

数値・記号・リストの分類

リスト操作

式の分解

再帰関数

式と環境の探索

入出力

結果と診断の表示

型判定

数値を受け取ったらそのまま返す。

記号なら環境を検索する。

リストなら先頭の構文を調べる。空の式の扱いも決める。

環境をデータにする

```
(quote ((x 3) (y 4)))
```

名前と値の組を並べる表現の一例。

参照実装の形式を確認し、検索関数と更新関数で隠す。

環境の抽象化

評価器の各分岐が、環境の内部構造を直接操作すると変更が広がる。

`lookup`、`define`、`extend` のような操作へまとめる。

辞書かリストかを、呼出し側から切り離す。

ホストの名前との区別

対象の `(define x 3)` は対象環境を更新する。

評価器実装の変数 `x` とは別の束縛。

両者を混ぜると、対象プログラムが評価器の内部名を参照できてしまう。

評価器の入力

入力には文字列と、読取済みのS式の2種類がある。

Lisp製評価器の中心にはS式を渡す。

文字列をS式にする入口は別に用意する。

数値の受入例

```
(define number-eval  
  (lambda (expr)  
    (if (number? expr) expr  
        (error "expected number"))))  
(number-eval 7)
```

説明用の最小例。未定義記号を黙って返さない。

quoteの受入例

対象式 `(quote (+ 1 2))` の結果はリスト `(+ 1 2)`。

対象式をホストに渡すときにも引用が必要になる。

どの層が引用を処理するかを図に書く。

二重の引用

```
(quote (quote (+ 1 2)))
```

外側の `quote` はホストが処理する。

残った `(quote (+ 1 2))` を対象の評価器が処理する。

ifの受入例

対象式 `(if #f (/ 1 0) 7)` は `7` を返す。

ホストで条件を評価してしまうと、対象環境の名前を扱えない。

条件も対象評価器へ渡す。

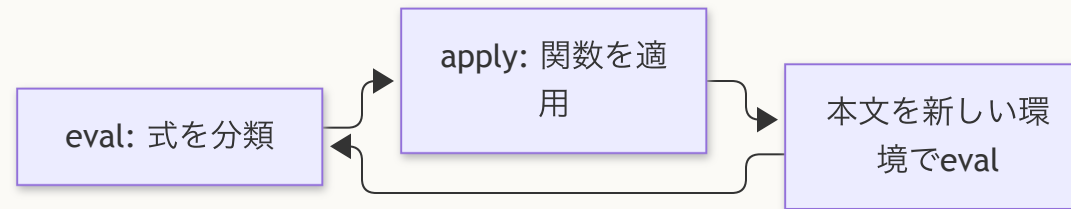
通常の呼出し

対象の先頭式を評価して関数を得る。

対象の引数式をそれぞれ評価する。

得た関数と値を `apply` の処理へ渡す。

evalとapply



式の規則と、関数の規則を分ける。

組込み手続き

算術や比較は、ホストの組込み手続きへ委譲できる。

委譲する名前と、引数・結果の型を決める。

未知の組込み名を黙ってホストへ渡さない。

対象のクロージャ

対象関数にも、仮引数、本文、定義時環境が必要。

ホストの `lambda` と対象の `lambda` を区別して表す。

対象関数を呼ぶとき、その環境を拡張する。

保存と共有

再帰関数は、自身の定義を後から環境で見つける必要がある。

環境を単なるコピーにするか、共有するかで振る舞いが変わる。

`fact` を受入例にして判断する。

完成参照の実演

```
cd examples/12-meta-evaluator  
python3 lisp.py --meta --eval "(+ 1 (* 2 3))"
```

`evaluator.lisp` の `m-eval` が計算し、結果は `7`。

第12回の対象機能は数値・四則・引用・分岐が中心。

開発順序

1. 数値と `quote`
2. 名前の検索と定義
3. `if` と組み込み呼出し
4. 対象の `lambda` と関数適用

各段階に正常例と反例を付ける。

演習1 機能契約

必要な操作を1個選び、入力・出力・失敗条件を書く。

その契約を使う対象式を示す。

便利だから追加する、という説明だけでは足りない。

演習2 最小評価器

数値と二項加算が通る入口を作る。

次に未定義名と未知の演算子を試す。

どの機能をまだ扱えないか明記する。

演習3 段階拡張

対象の `if` を追加し、非選択枝のゼロ除算を避ける。

名前と関数の契約は設計し、第13～14回で統合する。

ホスト環境との混同を検出する。

確認問い

なぜホストの評価関数へ対象式を渡すだけでは不十分か。

組込みを委譲するとき、Lispで実装した範囲をどう説明するか。

実装の境界

現段階はPython製Lispホスト上で動くLisp製評価器。

読取・数値表現・OS入出力などのホストへの依存が残る。

ネイティブ起動や独立ブートストラップの達成とは区別して報告する。

まとめと次回

式、環境、関数の契約をLispのデータと処理で表した。

次回は実装の理解を、予測と反例の議論で確認する。

参考

- 本教材: `examples/12-meta-evaluator/`
- SICP メタ循環評価器
- The Scheme Programming Language

参照資料と教材方言の機能差は仕様書で確認する。